

SUD: SECURE UPDATE DOWNLOADER

1. EXECUTIVE SUMMARY

SUD (Secure Update Downloader) is a secure software update and distribution system designed to let end users install and update applications from any network, while giving developers a controlled, signed update channel. It consists of:

- **SUD Manager** (client) for users: install, update, register, and manage downloads.
- **SUD Developer** (publisher) for developers: generate keys, sign manifests, package installers,

The security model relies on public key identity, signed manifests, and SHA-256 file integrity checks, with optional key rotation that preserves update trust over time.

This report documents the full technical architecture, recent feature additions (application registration and install path modification), engineering challenges encountered, and the planned roadmap for future development.

2. WHAT IS SUD

SUD is a software distribution and update platform with three core goals:

1. Manage app downloads and installations (local or remote).
 2. Automatically update apps when new versions are available.
 3. Ensure updates are authentic and untampered, even on untrusted networks.
-

3. SYSTEM ROLES

3.1 SUD MANAGER (CLIENT)

SUD Manager is the end-user application that acts as the secure update gate on the client device. It provides:

- A download manager with pause/resume/cancel/delete and persistent state.
- Installation and update workflows for `.exe`, `.msi`, `.pkg`, `.zip` packages.
- Background downloads with observer-driven install and update continuation.
- Application registration from manifest (local file or remote URL) without requiring a full install.
- Install path modification for registered or installed applications.
- Identity trust and fingerprints for first-time installs.
- Status checks and update availability scans.
- Local audit logs and app status tracking.
- CLI and GUI access to the same service layer.

For clients, SUD reduces update risk and manual effort. It allows updates from any network (even HTTP) while keeping security guarantees through signatures and hashes.

3.2 SUD DEVELOPER (PUBLISHER)

SUD Developer is the tooling for app publishers. It provides:

- Key generation (Ed25519) and key management.
- Manifest generation and signing.
- Packaging into `.pkg` (zip-based) format.

4. ARCHITECTURE AND CORE CONCEPTS

4.1 APPLICATION ARCHITECTURE

SUD follows a layered architecture with clear dependency direction:

- **Presentation layer:** CLI commands (`presentation/cli`) and GUI (`presentation/ui/tk`). The GUI includes Applications, Downloads, and Logs sections and a CLI input box.
- **Application layer:** orchestrates workflows via service classes:
 - `InstallAppService` — full install from manifest/package/URL
 - `UpdateAppService` — secure update with version and signature checks
 - `CheckUpdatesService` — lightweight status check against update URLs
 - `DownloadFileService` — multi-segment download with observer automation

- `RegisterAppService` — register an app identity without full installation
- `ModifyAppPathService` — change the install path of a registered app
- `UninstallAppService` — uninstall and cleanup
- `ListAppsService` — list apps with optional live status check
- `ListLogsService` — retrieve update audit logs
- **Domain layer:** pure rules and models (`Manifest`, `Version`, `IdentityService`, `VerificationService`, `VersionService`).
- **Infrastructure layer:** network, downloader (aria2 RPC), package extraction, installer strategies, cryptography, and persistence (SQLite via `AppRepository`).

Presentation depends on application services, which use domain rules and call into infrastructure. This keeps security rules centralized and keeps I/O replaceable.

4.2 DOMAIN EXCEPTION HIERARCHY

SUD defines a structured exception hierarchy rooted at `SudError`:

Exception	Purpose
<code>UserRejectedError</code>	User explicitly rejected a trust prompt
<code>SignatureVerificationError</code>	Cryptographic signature validation failed
<code>HashMismatchError</code>	File hash does not match manifest value
<code>VersionRollbackError</code>	Update attempts to install older version
<code>AppNotFoundError</code>	Application not found in local repository
<code>NetworkError</code>	Network operations (download/fetch) failed
<code>PackageError</code>	Package extraction or manifest loading failed
<code>InstallerError</code>	Installer execution or uninstallation failed
<code>PathDoesNotExistError</code>	Specified filesystem path does not exist

4.3 TEST BACKEND ARCHITECTURE (LOCAL UPDATE SERVER)

The `backend` folder contains a small Express server used for local testing:

- `/updates/from/:manifestName` serves manifests using version routing logic.
 - `manifest0.0.0` → `manifest_1.0.0.json` (bootstrap install)
 - `manifest1.0.0` → `manifest_1.0.1.json` (normal update)
 - `manifest1.0.1` → `manifest_1.0.0.json` (rollback simulation)
- `/exe/:fileName` serves installer binaries from `backend/files`.
- Throttling is enabled at 2 MB/s to simulate constrained networks.

This backend is a test harness, not a hardened production service.

4.4 MANIFEST

The manifest is the signed update contract between developer and client. It includes:

Field	Description
<code>app_id</code>	Unique label used by UI and CLI
<code>app_name</code>	Human-friendly name
<code>version</code>	Semantic version string
<code>public_key</code>	App identity and trust anchor
<code>signature</code>	Signature over canonical JSON payload
<code>file_hash/installer_hash</code>	SHA-256 of the installer binary or package
<code>file_source/installer_url</code>	URL to download the installer
<code>update_urls</code>	List of URLs used to fetch future manifests
<code>file</code>	Installer filename inside a package or name for direct download

Manifest parsing normalizes fields (`file_hash` vs `installer_hash`, `file_source` vs `installer_url`) and accepts base64 or hex signatures, which are converted into bytes.

4.5 APP IDENTITY

The true identity of an app is the **public key**, not the `app_id` or `app_name`. This prevents attackers from spoofing an app by reusing a name or ID. Fingerprints are generated for user trust prompts.

4.6 STORAGE AND PERSISTENCE

SUD Manager uses SQLite (`app.db`) via SQLAlchemy. Key tables:

`apps` table:

Column	Type	Notes
id	Integer PK	Auto-increment
app_id	String	Application identifier
app_name	String	Display name
public_key	String (unique, indexed)	Identity anchor
version	String	Current version
update_urls	JSON	List of manifest URLs
install_path	String	Filesystem path (mutable via <code>ModifyAppPathService</code>)
status	String	up to date, update available, unreachable, unknown
created_on	DateTime	Record creation timestamp
updated_on	DateTime	Last modification timestamp

`downloads` table: download metadata, progress, aria2 GID, observer command.

`logs` table: update history (version, prior keys, status, etc).

Persistence enables resuming downloads and remembering trusted identities across restarts.

5. NEW FEATURES (LATEST MODIFICATIONS)

5.1 REGISTER APP SERVICE (`REGISTERAPPSERVICE`)

Purpose: Allow users to register an application's identity and metadata from a manifest *without* performing a full installation. This is useful when:

- The application was installed manually outside of SUD.
- The user wants to track updates for an app they already have.
- The user wants to establish trust before downloading.

Workflow:

1. Load manifest (from local file path OR remote HTTP URL)
2. Check if `public_key` already exists in DB
 - └ YES → Return "already registered" (idempotent)
 - └ NO → Continue to trust flow
3. Generate fingerprint from `public_key`
4. Display app info + fingerprint to user
5. Ask user for explicit trust confirmation
 - └ Rejected → raise `UserRejectedError`
 - └ Accepted → Continue
6. Verify manifest signature
 - └ Invalid → raise `SignatureVerificationError`
 - └ Valid → Continue
7. Store app in DB with:
 - `app_id`, `app_name`, `public_key`, `version`, `update_urls`
 - `install_path` (optional, provided via `--path` flag)
 - `status`: "up to date" if path provided, "unknown" otherwise
8. Return success with advisory message if no path was given

CLI Usage:

```
# Register from local manifest
python main.py register ./manifest.json --path "C:\Program
Files\MyApp"

# Register from remote manifest
python main.py register https://example.com/manifest.json

# Register without specifying install path (can be added later)
python main.py register ./manifest.json
```

Design Decisions:

- The service accepts both local file paths and HTTP URLs, using `NetworkService.fetch_manifest()` for remote sources.
- Registration is **idempotent**: re-registering an app with the same public key silently succeeds.
- The `--path` flag is optional — users can register without knowing the install location and add it later via `modify`.

- When no path is provided, status is set to "unknown" to signal incomplete registration.

5.2 MODIFY APP PATH SERVICE (MODIFYAPPPATHSERVICE)

Purpose: Allow users to change the recorded install path for a registered application. This addresses scenarios where:

- The app was registered without a path and the user wants to add one.
- The user moved the application to a different directory.
- The original path was incorrect.

Workflow:

```
1. Validate that new_path exists on filesystem
   └ Does not exist → raise PathDoesNotExistError
2. Look up app by app_id
   └ Not found → raise AppNotFoundError
3. Update install_path in DB
4. Set status to "up to date"
5. Commit transaction
6. Return success message
```

CLI Usage:

```
python main.py modify sud_calculator "C:\Program Files\Calculator"
```

Design Decisions:

- Path existence is validated at service level before any DB mutation, preventing dangling references.
- The command joins remaining CLI arguments with spaces to support paths containing spaces (e.g., C:\Program Files\...) without requiring quotes in all shells.
- Status is reset to "up to date" after path modification, as the app is presumed correctly installed at the new location.

5.3 UPDATED CLI COMMAND SET

The full command set, including new additions, registered in the `CommandRouter`:

Command	Service	Description
<code>install <path_or_url></code>	<code>InstallAppService</code>	Install from manifest, package, or URL
<code>update <app_id></code>	<code>UpdateAppService</code>	Check and apply updates
<code>uninstall <app_id></code>	<code>UninstallAppService</code>	Remove app and metadata
<code>list</code>	<code>ListAppsService</code>	List all registered apps
<code>status <app_id></code>	<code>CheckUpdatesService</code>	Check for available updates
<code>download <url></code>	<code>DownloadFileService</code>	Download a file with full management
<code>logs</code>	<code>ListLogsService</code>	View update audit logs
<code>register <manifest> [--path]</code>	<code>RegisterAppService</code>	Register app identity (<i>new</i>)
<code>modify <app_id> <path></code>	<code>ModifyAppPathService</code>	Change install path (<i>new</i>)
<code>ui</code>	—	Launch Tkinter GUI

6. DOWNLOAD SUBSYSTEM (SUD MANAGER)

6.1 MULTI-SEGMENT DOWNLOADER

SUD delegates multi-segment downloading to aria2c via RPC. Segmentation happens inside aria2c:

- `max-connection-per-server = 5` (parallel connections per host).
- aria2 automatically splits a file into ranges when the server supports HTTP Range.

6.2 PAUSE / RESUME / CANCEL / DELETE

The `DownloadManager` maps each download to an aria2 GID and stores state in the database.

- **pause**: calls `aria2.pause(gid)` and updates status to paused.
- **resume**: calls `aria2.unpause(gid)`; if the GID is missing, it re-adds the URL.
- **cancel**: removes the download from aria2 and deletes partial files.
- **delete**: cancels the download and removes the DB record.

6.3 RESUME AFTER RESTART

Downloads persist in SQLite and can be resumed later:

1. The app restarts and reads the download record from DB.
2. If the previous aria2 GID is still valid, aria2 continues tracking progress.
3. If the GID is missing or invalid, the downloader re-adds the URL and uses the same file path, allowing aria2 to resume from the partial file.

6.4 OBSERVER CALLBACK (AUTOMATION)

When a download finishes, SUD triggers an observer command. This is used to:

- Start an install after a package download.
- Resume update flow with a downloaded file path.

The observer command supports a `%PATH%` placeholder that is replaced with the downloaded file's actual path at runtime.

6.5 FILENAME AND EXTENSION GUESSING

If the caller does not provide a file name, SUD derives it from the URL path:

1. Parse URL path and take the last segment.
2. Strip invalid filesystem characters.
3. If empty, use `"download"`.
4. If no extension exists, append `".bin"`.

7. PACKAGE SUBSYSTEM (.PKG, .ZIP, .RAR)

7.1 SUPPORTED FORMATS

- `.pkg` (zip-based)
- `.zip`
- `.rar` (requires a module inside client PC)

7.2 SAFE EXTRACTION

Zip and RAR extraction implement a path traversal check. Any archive entry that would escape the destination folder is rejected. This protects against zip-slip attacks.

7.3 PACKAGE STRUCTURE

A valid package must include: `manifest.json` and the installer binary referenced by `manifest.file`.

8. INSTALLER SUBSYSTEM (SUD MANAGER)

8.1 STRATEGY PATTERN

Installers are executed using a Strategy Pattern. Supported strategies:

Strategy	Flags	Path Parameter
MSI (msiexec)	—	INSTALLDIR / TARGETDIR
Inno Setup	/VERYSILENT or /SILENT	/DIR=<path>
NSIS	/S	/D=<path> (must be last, unquoted)
InstallShield	/s	/v"INSTALLDIR=..."
WiX Burn	/install /quiet /norestart	InstallFolder
Interactive	—	Manual GUI install (fallback)

8.2 INSTALLER SELECTION

1. `manifest.file` (explicit).
2. First executable or `.msi` in the package if not specified.

8.3 UNINSTALL FLOW

Uninstall attempts to locate common uninstallers (`uninstall`, `unins`, `remove`) and runs them with a timeout. If forced, failure can be ignored.

9. INSTALL FLOW (SUD MANAGER)

9.1 REMOTE MANIFEST INSTALL

If the install path is an HTTP URL and resolves to a manifest:

1. Fetch manifest via `NetworkService`.
2. Verify manifest signature using its `public_key`.
3. Start download of `manifest.installer_url` using `DownloadManager`.
4. Attach an observer to trigger install after download.

First-install trust rationale: at this stage the client does not yet have a stored public key, so the manifest's own `public_key` is used to verify its signature. The user is shown the fingerprint and must explicitly trust it; only after acceptance is the public key stored for all future updates. This way we can verify that the file and manifest came from the same person and the public key provided is really his public key

9.2 DIRECT HTTP PACKAGE OR BINARY

If the URL is not a manifest:

- Start download with observer to trigger install once complete.

9.3 LOCAL PACKAGE (.PKG/.ZIP/.RAR)

1. Extract package to temp directory.
2. Load `manifest.json`.
3. If app is new, show fingerprint and ask for user trust.
4. Verify signature and installer hash.
5. Run installer and store app metadata.

9.4 LOCAL BINARY (.EXE/.MSI)

If installing from a direct binary, SUD requires a previously provided manifest and uses it to validate the hash before running the installer.

10. UPDATE FLOW (SUD MANAGER)

10.1 STATUS CHECK

`CheckUpdatesService`:

1. Load installed app from DB.
2. Fetch manifest from `update_urls`.
3. Compare versions using SemVer rules.
4. Verify manifest signature with the **stored** public key.
5. Mark app status as "update available" or "up to date".

10.2 UPDATE EXECUTION

`UpdateAppService` performs a strict security sequence:

1. Fetch manifest and compare versions.
2. Verify manifest signature with stored public key.
3. Key rotation: if `manifest.public_key` differs, accept only after the old key validated the manifest.
4. Download installer or package (optional background download using observer).
5. Verify hash and signature:
 - For `.exe/.msi`: verify hash against `manifest.installer_hash`.
 - For `.pkg/.zip`: verify package hash, extract, then verify inner manifest signature and inner installer hash.
6. Update DB metadata, add audit log, and run installer.

This pipeline blocks updates if signature or hash validation fails. DB changes are wrapped in a transaction with rollback on failure.

11. SECURITY MODEL AND ALGORITHMS

11.1 PUBLIC KEY IDENTITY

The public key is the canonical identity. This prevents spoofed app IDs or names.

11.2 SIGNATURE VERIFICATION

Manifests are signed using canonical JSON (sorted keys, compact separators). Supported algorithms:

- **Ed25519** (preferred) for compact, modern signatures.
- **RSA with PKCS1v15 and SHA-256** as fallback.

11.3 HASH VERIFICATION

Installer and package files are verified with **SHA-256**.

11.4 VERSION COMPARISON (ROLLBACK PROTECTION)

SemVer (Semantic Versioning) is used: `MAJOR.MINOR.PATCH`. Updates are allowed only when `new_version > current_version`.

11.5 FINGERPRINTS AND TRUST PROMPTS

Public key fingerprint (SHA-256) is shown to the user at first install or registration. Trust is required before the app is stored.

11.6 SAFE EXTRACTION

Archive extraction checks for path traversal to block zip-slip attacks.

12. THREAT MODEL AND MITIGATIONS

Threat	Mitigation
MITM on update networks	Signed manifest + SHA-256 hash verification. Tampering is detected unless attacker has the private key.
Malicious package tampering	Outer package hash + inner manifest signature + inner installer hash.
Downgrade/rollback attacks	SemVer comparison blocks <code>new_version <= current_version</code> .
Identity hijacking / key rotation abuse	New key accepted only after manifest is verified by the previous trusted key.
Zip-slip (path traversal)	Safe extraction checks prevent writing outside extraction directory.
Timing attacks on identity comparison	<code>hmac.compare_digest</code> used for constant-time key comparison.

13. DEVELOPER INTEGRATION GUIDE (SUD DEVELOPER)

13.1 GENERATE KEYS

Use `generate_manifest.py` (CLI) or `sud_developer_ui.py` (GUI). Both generate Ed25519 key pairs.

13.2 BUILD AND SIGN A MANIFEST

Required fields: `app_id`, `version`, `file`, `file_hash`, `public_key`, `signature`. Signing uses canonical JSON (sorted keys, compact separators). Signature stored as base64.

13.3 PACKAGE THE APP

Create a folder with the installer binary, add `manifest.json`, zip and rename to `.pkg`.

13.4 HOST UPDATE ENDPOINTS

Host manifest files and installers over HTTP/HTTPS. Each manifest can point to a later manifest using `update_urls` for stable update links.

14. TESTING AND VALIDATION

14.1 PREREQUISITES

- Python dependencies: `pip install -r requirements.txt`
- aria2c on PATH (included by default in the application folder)
- Node dependencies for backend: `npm install` (inside backend folder)

OR

- Use compiled version

14.2 LAUNCH THE TEST BACKEND

```
cd backend
npm install
node server.js
```

Expected: server runs on `http://localhost:8000`.

14.3 INSTALL USING MANIFEST0.0.0 (CLI)

```
python main.py install
http://127.0.0.1:8000/updates/from/manifest0.0.0.json
```

14.4 UPDATE TO 1.0.1 (CLI)

```
python main.py update sud_calculator
```

14.5 ROLLBACK ATTEMPT (1.0.1 TO 1.0.0)

```
python main.py update sud_calculator
```

Expected: "No update" — version stays at 1.0.1.

14.6 REGISTER AN APP (CLI)

```
# Register from remote manifest with install path
python main.py register
http://127.0.0.1:8000/updates/from/manifest1.0.0.json --path
"C:\SUD\sud_calculator"

# Register from local manifest without path
python main.py register ./manifest.json
```

14.7 MODIFY INSTALL PATH (CLI)

```
python main.py modify sud_calculator "D:\Apps\Calculator"
```

14.8 UI STEPS (SUD MANAGER GUI)

1. Launch the UI: `python main.py ui`
2. Applications tab → Install App → paste the manifest0.0.0 URL.
3. Accept the trust prompt (first install).
4. Watch the Downloads tab for progress.
5. Applications tab → right click → Check for updates → accept update.
6. Repeat the update check after 1.0.1; it should report no update available.

The UI includes a CLI input box at the bottom that can run the same commands.

14.9 UNIT TEST COVERAGE

The `tests/` directory includes unit tests for:

Test File	Coverage Area
<code>test_crypto_service.py</code>	Cryptographic signing/verification
<code>test_download_service.py</code>	Download manager operations
<code>test_identity_service.py</code>	Fingerprint generation
<code>test_install_app_integration.py</code>	End-to-end install flow
<code>test_install_service.py</code>	Installer strategy detection
<code>test_network_service.py</code>	Manifest fetching
<code>test_package_service.py</code>	Package extraction
<code>test_storage_service.py</code>	Repository CRUD operations
<code>test_uninstall_cmd.py</code>	Uninstall CLI command
<code>test_verification_service.py</code>	Signature + hash verification
<code>test_version_service.py</code>	SemVer parsing and comparison

15. ENGINEERING CHALLENGES

15.1 OBSERVER COMMAND PATH HANDLING

Problem: When a download completes, the observer command needs to reference the downloaded file path. Paths on Windows contain spaces (e.g., `C:\Program Files\...`) and special characters that break shell argument parsing.

Solution: A `%PATH%` placeholder is injected into the observer command string and replaced at runtime with the actual file path, wrapped in quotes. The observer is routed back through the `CommandRouter` directly (in-process) rather than spawning a subprocess, avoiding shell escaping issues entirely.

15.2 MANIFEST SOURCE FLEXIBILITY IN REGISTRATION

Problem: The `RegisterAppService` needed to accept manifests from both local files and remote URLs, each requiring different loading logic. Remote manifests return a `Manifest` object directly from `NetworkService`, while local files require JSON parsing and `Manifest.from_dict()`.

Solution: The service detects the source type via URL prefix check (`http://` / `https://`) and branches accordingly, with unified error handling wrapping both paths in a single `PackageError`.

15.3 PATH VALIDATION ACROSS SERVICES

Problem: Multiple services (`RegisterAppService`, `ModifyAppPathService`, `InstallAppService`) need to validate filesystem paths, but validation rules differ — some require the path to exist, others create it.

Solution: A dedicated `PathDoesNotExistError` exception was added to the domain exception hierarchy. Each service validates paths at its own boundary before mutating state, following the fail-fast principle.

15.4 DUAL MANIFEST VERIFICATION IN PACKAGE UPDATES

Problem: When updating via a `.pkg` package, two manifests exist: the remote manifest (manifest A, fetched from `update_urls`) and the inner package manifest (manifest B, inside the `.zip`). Both must be verified, but against different keys and for different purposes.

Solution: Manifest A is verified against the **stored** public key (trust chain). Manifest B is verified using its **own** public key (self-consistency). The package hash is checked against manifest A, and the inner installer hash is checked against manifest B. This dual-layer verification prevents both network-level and package-level tampering.

15.5 IDEMPOTENT REGISTRATION

Problem: Users might attempt to register the same app multiple times, which could create duplicate entries or confuse the trust model (since identity is based on `public_key` uniqueness).

Solution: `RegisterAppService` checks for an existing app by `public_key` before proceeding. If found, it returns a success response with `is_new_app: False`, making registration idempotent without errors.

15.6 DOWNLOAD RESUME ACROSS RESTARTS

Problem: aria2c GIDs are ephemeral — they are lost when the aria2c daemon restarts. Downloads tracked in the database cannot be blindly resumed with stale GIDs.

Solution: The resume flow checks GID validity first. If the GID is invalid, the URL is re-added to aria2c with the same file path. aria2c detects the existing partial file and resumes from where it left off, provided the server supports HTTP Range headers.

15.7 UI FRAMEWORK MIGRATION

Problem: The original UI was built with Flet, which introduced dependency and packaging complexities. The project needed a lighter, more portable GUI framework.

Solution: The UI was migrated from Flet to Tkinter, which is included in Python's standard library. This eliminated external GUI dependencies and simplified packaging into standalone executables via PyInstaller/Nuitka.

16. OPERATIONAL NOTES AND LIMITATIONS

- Resuming partial downloads depends on server support for HTTP Range.
- The update server example is a reference implementation, not a hardened service.
- HTTPS is recommended even though signatures protect integrity, to reduce metadata leakage.
- **HTTPS is required for first install.** There is no way to distinguish a real manifest from a fake one on first contact when no trusted key is stored locally.

17. FUTURE ROADMAP

17.1 SHORT-TERM (NEXT RELEASE)

Item	Description
Mirror support in manifests	Add <code>mirrors</code> field to manifests so downloads can fallback to alternate URLs automatically. Currently stubbed as <code>None</code> in <code>InstallAppService</code> .
CommandRouter input validation	Implement the TODO for validating <code>argv</code> length and providing usage hints for malformed commands.
Register from UI	Expose the <code>register</code> and <code>modify</code> commands through the Tkinter GUI, not just CLI.
Fix <code>`UninstallAppService`</code> naming	Rename to <code>UninstallAppService</code> across the codebase with a migration step.):

17.2 MEDIUM-TERM

Item	Description
Automatic scheduled status checks	Background service that periodically polls <code>update_urls</code> and notifies the user of available updates.
Differential / delta updates	Instead of downloading full installers, compute and apply binary diffs to reduce bandwidth.
Multi-app batch operations	<code>update --all, status --all, register --batch <file></code> for managing large app portfolios.
Manifest versioning (v2 schema)	Add schema version field to manifests for forward compatibility. Include <code>mirrors</code> , <code>changelog</code> , <code>min_os_version</code> .
Rollback with user consent	Allow deliberate downgrades when explicitly requested by the user, while maintaining the default block.

17.3 LONG-TERM

Item	Description
Plugin / extension system	Allow third-party installer strategy plugins for custom package formats.
Certificate-based trust	Support X.509 certificate chains as an alternative to raw public keys, enabling organizational trust hierarchies.
Cross-platform packaging	Extend installer strategies to cover macOS <code>.dmg/ .app</code> and Linux <code>.deb/ .rpm/ .AppImage</code> .
Manifest transparency log	Public append-only log of all published manifests (similar to Certificate Transparency) for auditability.
Sandboxed installation	Run installers in isolated environments to limit damage from compromised packages.
Auto-discovery protocol	Allow apps to advertise their SUD manifest URL via DNS TXT records or <code>.well-known</code> endpoints.

18. SUMMARY

SUD provides a complete secure update pipeline:

- **Signed manifests** establish authenticity.
- **SHA-256 hashes** validate installer integrity.
- **Public key identity** prevents spoofing.
- **Key rotation** is supported without losing trust.
- **aria2c** provides robust multi-segment downloads with pause/resume.
- **Packages** are safely extracted and verified.
- **Application registration** allows tracking apps installed outside SUD (*new*).
- **Path modification** enables flexible install location management (*new*).

This design allows automatic updates on any network while preserving security guarantees and auditability for both end users and developers.